

Adopting Lightspeed's DeepAgents runtime in Konveyor

November 11, 2026

Runtime integration

- Konveyor with Goose
- The prototype with Lightspeed
- Changes in Lightspeed's runtime
- Coolstore planning demo
- Questions

Konveyor with Goose

1. **You choose an app and a task**

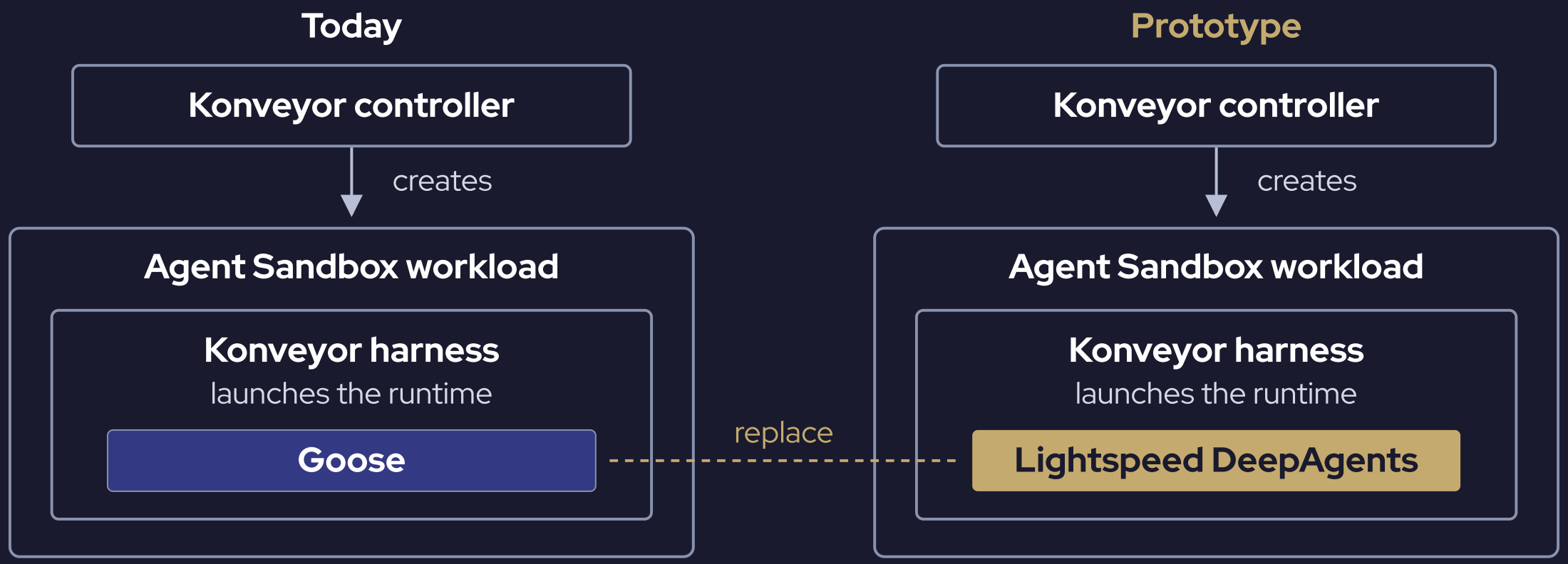
2. **Our harness prepares the repo**

3. **Goose works on the code**
   Calls the model, uses tools, commits locally

4. **Our harness verifies and pushes**
   You get the result on a Git branch



Konveyor with Lightspeed



Our harness still prepares the repo, verifies the work and pushes to Git.

Runtime changes and why we need them

- **Keep the conversation:** retain the graph and clients so the harness can request a handoff in a later prompt.
- **Reuse official ACP:** connect Lightspeed to our UI through `AgentServerACP`, while preserving user control.
- **Keep output inspection:** release tool results to viewers after inspection passes, including during replay.
- **Report execution accurately:** count model turns, report usage, apply model settings and detect incomplete responses.

The harness still owns Hub, verification and Git push.

A second prompt in the same conversation

Current upstream path: `provider.query()` creates a graph and model clients for one query, then closes the clients.

Change: `RuntimeSession` owns the graph, checkpoint history and clients until the session closes. Later prompts reuse them.

Why Konveyor needs it: after the primary prompt reaches `maxTurns`, the harness sends another `session/prompt` request to write a handoff before finalization.

Sources: current query path, session preparation

Lightspeed behind the official ACP server

Official ACP already provides: session requests, prompt handling and rendering of graph events into ACP updates.

Change: an optional adapter connects Lightspeed's prepared runtime to `AgentServerACP`. Lightspeed retains graph preparation, inspection and runtime controls.

Why Konveyor needs it: our harness and UI can keep using ACP while the runtime reuses the official protocol implementation.

Sources: official server, Lightspeed adapter

User control during execution

Permissions and questions: callbacks ask the host to approve a tool or answer the registered `ask_user` tool.

Guidance and cancellation: guidance enters at the next model boundary. Cancellation waits for active work to stop and ends local shell process groups.

Why Konveyor needs it: users already supervise runs through the UI. The runtime must return control while the harness can still finalize the workload.

Sources: callbacks and controls, local shell execution

Turn limits, usage and the handoff prompt

Current upstream path: returns usage totals after the query. Its graph recursion limit counts execution steps, which differ from model calls.

Change: report observed tokens as calls complete and count main-model calls against the prompt's turn allowance.

Why Konveyor needs it: show usage during the run, stop the primary prompt at its allowance, then send a handoff prompt using reserved turns.

Sources: usage reporting, turn counter

Inspection before model and viewer access

Lightspeed already checks: tool results for instructions that could redirect the agent, before the next model call.

Change: the ACP adapter publishes tool results only after Lightspeed completes inspection of that boundary. A cached classification alone does not authorize display.

Why Konveyor needs it: files and MCP output can contain hostile instructions. Switching transports must preserve both model inspection and approved viewer release.

Sources: Lightspeed inspection, approved output gate

Viewer replay without reopening rejected output

Official replay: reconstructs messages from checkpoints, which can retain tool content that Lightspeed never approved for display.

Change: journal authorized outgoing updates. The subclass overrides private `_replay_session()` to replay that journal to reconnecting viewers.

Why Konveyor needs it: reconnecting to a live run must preserve the same display boundary as watching it continuously.

Version coupling: five private overrides require a pinned `deepagents-acp` version and upgrade tests.

Sources: official replay, Lightspeed replay

Applying the model settings we choose

Current provider: Anthropic constructors forward thinking settings and output caps. The normalized reasoning-effort choice was not passed through.

Change: map effort to `output_config.effort` and allow an optional output-token limit for the main model.

Why Konveyor needs it: the GLM demo selected low effort and a 16,384-token response allowance. Those choices must reach the model request.

Sources: provider mapping, optional output allowance

Detecting a response that was cut short

Current path: records provider finish reasons for telemetry. An explicit output cutoff does not itself prevent tool execution.

Change: raise `ModelOutputLimitError` before executing tools from an unfinished response. Keep the usage already observed.

Why Konveyor needs it: a partial answer must not look like completed work. The harness still verifies the requested plan before pushing.

Sources: current finish-reason reporting, execution gate

Coolstore planning demo

A Java EE application we want to move to Quarkus

Play the separate demo video (2:44)

Recording: earlier custom ACP adapter at `4039163`.

Code and result:

- Konveyor harness integration: PR #345 · source changes
- Recorded runtime: custom ACP at 4039163
- Updated runtime: official ACP
- Konveyor UI changes: djzager/tackle2-ui
- Reviewed plan: djzager/coolstore, docs/plan.md

Questions

Lightspeed as the execution backend

Today: Konveyor's `AgentRun` reconciler creates a Sandbox directly. Our Go harness owns preparation, agent execution and finalization.

Candidate integration: delegate execution to a Lightspeed `AgenticRun`; its operator owns the SandboxClaim and workload lifecycle.

Adoption bar: per-run image/startup control, complete access to inputs and credentials, and preservation of harness finalization and outcomes.

Agent Sandbox remains underneath Lightspeed's SandboxClaim path. ADR 0021 separates this qualification from runtime reuse.

Roadblock: selecting the workload image

Current: one shared `sandbox-pod-spec` supplies the base image. Agent and AgenticRun have no per-invocation image selector.

- Java EE to Quarkus needs a JDK and Maven in the workload.
- A future Go Migrator needs the Go toolchain.
- Both images must be usable in concurrent runs.

Required: select the image per invocation and start its Konveyor harness.

Possible contract: a per-run image override or template selection, honoring the image's entrypoint. ADR 0021 makes image/startup control a firm requirement.

Problem: workload placement and access

Current: Lightspeed watches runs across namespaces but creates workloads in its operator namespace. Agent/LLMProvider are cluster-scoped; Secrets are namespaced.

Problem: the workload needs Hub's run-scoped inputs/token, model credentials and a reachable ACP endpoint for Konveyor's UI.

Options to qualify: run near Hub and bind provider credentials there, or run centrally and deliver Hub inputs/token with the required network access.

Components: Hub, provider/Secret binding, PodSpecBuilder and workload networking. ADR 0021 leaves placement open.

Problem: harness input and run completion

Current: Lightspeed mounts batch inputs and a Result template under `/input`; its operator uses the Pod and a completed Kubernetes Result to determine success.

Adapter needed: our harness must consume those inputs, prepare the repo, run the agent, verify/push, clean up and publish the backend Result before exiting.

Why: an agent finishing its answer is earlier than the workload finishing.

If push succeeds but Result publication fails, preserve the work and report the integration failure. ADR 0021 requires that distinction.

Problem: stopping the agent or its workload

Runtime cancellation: stops active prompt/tool work inside the Pod; the Go harness can still finalize.

Operator hard stop: terminates the workload and may prevent handoff, verification, push, token cleanup and Result publication.

Needed: an agreed normal stop path and time to finalize within the workload deadline, with emergency termination handled explicitly.

Discussion: can operator cancellation request harness wind-down before killing the workload? A per-run hard-stop proposal is open in PR #588.

OpenShell roadmap and integration contract

Roadmap question: are you planning OpenShell in this execution path, and on what timeline?

If so: which layer would own workload image/startup, placement, network policy and credential binding?

Integration impact: Hub/Git/model/MCP/ACP access and Kubernetes Result publication need a supported contract under that backend.

Should we qualify today's batch contract, or align on one that remains stable through that change?

ADRs 0016 and 0021 keep OpenShell deferred and preserve today's Gateway selection contract.

Questions